# NYISO Day-Ahead Electricity Prices

How do prices vary over time, and when are unusually high prices concentrated?

Source: NYISO OASIS day-ahead zonal LBMP CSV | N.Y.C. | PTID 61761
8,760 hourly records | 2025-06-18 to 2026-06-17 | USD/MWh

### 1 Parse, validate, and summarize

```
# Hourly NYISO prices offer a real-world case for DST-aware time-series analysis.
# This workflow combines data cleaning, custom functions, and price visualization.
from pathlib import Path
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

csv_path = Path(__file__).with_name("nyiso_day_ahead.csv")
FIELDS = {"Eastern Date Hour": "wall", "Zone Name": "zone",
          "Eastern Date Hour Time Zone": "tz", "Zone PTID": "ptid",
          "DAM Zonal LBMP": "price"}

def clean_prices(raw):
    df = raw.rename(columns=FIELDS)[list(FIELDS.values())].copy()
    wall = pd.to_datetime(df["wall"], format="%Y/%m/%d %H:%M",
                          errors="coerce")
    shift = df["tz"].map({"EDT": 4, "EST": 5})
    df["price"] = pd.to_numeric(df["price"], errors="coerce")
    df["ptid"] = pd.to_numeric(df["ptid"], errors="coerce")
    if wall.isna().any() or shift.isna().any() or df.isna().any().any():
        raise ValueError("Missing field, invalid time/number, or unknown zone.")
    if not np.isfinite(df[["price", "ptid"]].to_numpy()).all():
        raise ValueError("Non-finite numeric value.")
    # EDT/EST disambiguates the repeated fall-back local hour.
    df["utc"] = (wall + pd.to_timedelta(shift, unit="h")).dt.tz_localize("UTC")
    df["local"] = df["utc"].dt.tz_convert("America/New_York")
    if not df["local"].dt.tz_localize(None).equals(wall):
        raise ValueError("Time-zone label disagrees with New York time.")
    df = df.sort_values("utc").reset_index(drop=True)
    if df["utc"].duplicated().any():
        raise ValueError("Repeated UTC instant; inspect the source revisions.")
    if not df["utc"].diff().dropna().eq(pd.Timedelta(hours=1)).all():
        raise ValueError("Hourly gap; inspect the source before analysis.")
    return df  # Keep all valid observations, including price spikes.

def summarize_prices(df):
    price = df["price"]
    stats = price.agg(["mean", "median", "max"]).to_dict()
    stats["p95"] = price.quantile(0.95)  # pandas linear interpolation
    # Simple hourly averages within each New York local calendar day.
    daily = price.set_axis(df["local"]).resample("D").mean().to_frame("mean")
    # Use a full seven-day window; leave the first six values undefined.
    daily["ma7"] = daily["mean"].rolling(7, min_periods=7).mean()
    # Year-month keys keep the two partial June months separate.
    grouped = df.assign(month=df["local"].dt.strftime("%Y-%m"),
                        hour=df["local"].dt.hour)
    heat = grouped.pivot_table(index="month", columns="hour",
                               values="price", aggfunc="mean")
    return stats, daily, heat
```

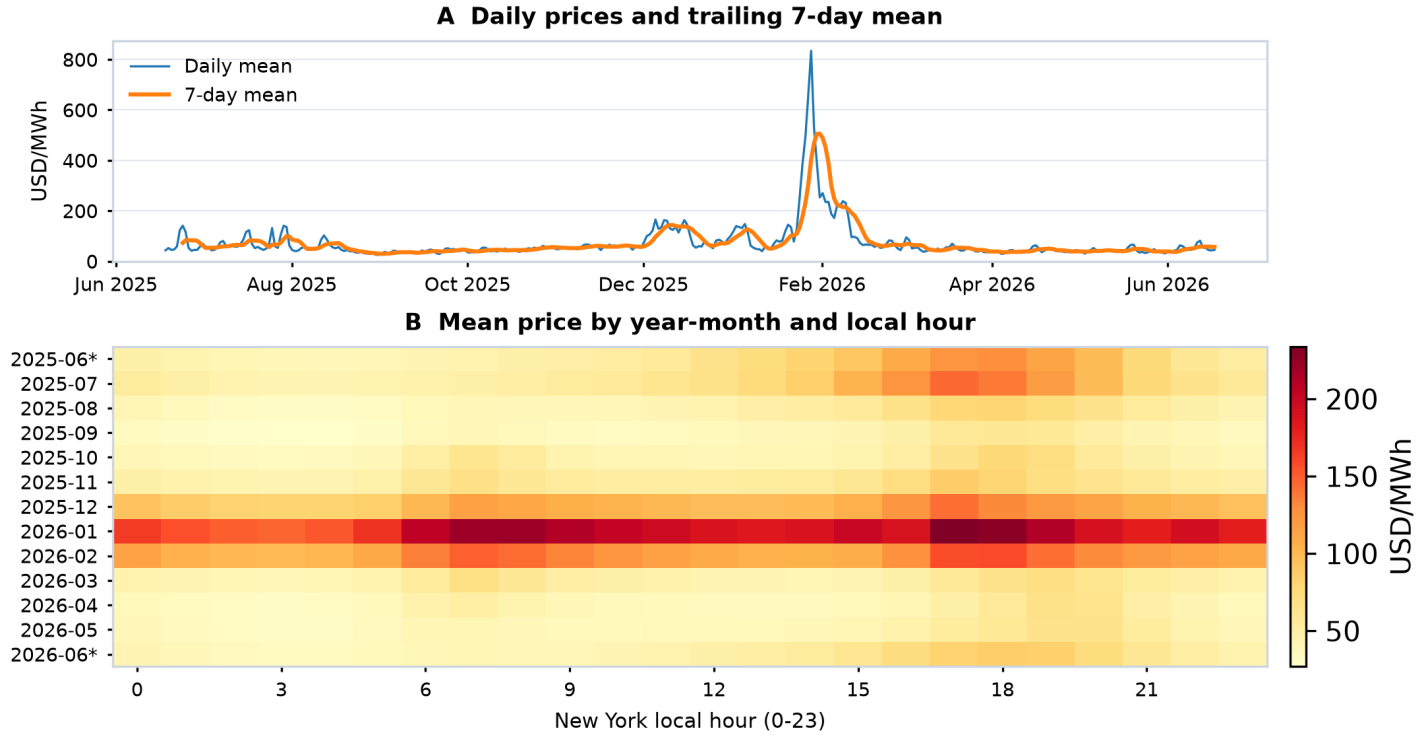
All valid prices retained. UTC validates hourly continuity; local time defines daily/hourly groups.

## Run the functions and inspect the output

```
raw = pd.read_csv(csv_path)
df = clean_prices(raw)
stats, daily, heat = summarize_prices(df)
high = df["price"] > stats["p95"]  # Strict, full-sample descriptive cutoff.
print("Mean/median/p95/max (USD/MWh): " +
      "/".join(f"{stats[k]:.3f}" for k in ["mean", "median", "p95", "max"]))
print(f"High-price hours: {high.sum():,}/{len(df):,} ({high.mean():.1%})")
winter = df["local"].dt.strftime("%Y-%m").isin(["2026-01", "2026-02"])
print(f"Jan-Feb 2026: {(high & winter).sum()}/{high.sum()} high-price hours")
fig, axes = plt.subplots(2, 1, figsize=(7.2, 3.75), layout="constrained",
                         gridspec_kw={"height_ratios": [1, 1.45]})
axes[0].plot(daily.index, daily["mean"], lw=0.8, label="Daily mean")
axes[0].plot(daily.index, daily["ma7"], lw=1.5, label="7-day mean")
axes[0].set(title="A  Daily prices and trailing 7-day mean",
            ylabel="USD/MWh")
axes[0].legend(fontsize=7, loc="upper left", frameon=False)
image = axes[1].imshow(heat, aspect="auto", cmap="YlOrRd")
axes[1].set(title="B  Mean price by year-month and local hour",
            xlabel="New York local hour (0-23)",
            xticks=np.arange(0, 24, 3), yticks=np.arange(len(heat)),
            yticklabels=heat.index)
fig.colorbar(image, ax=axes[1], label="USD/MWh", pad=0.02)
```

### ACTUAL RUN OUTPUT

```
Mean/median/p95/max (USD/MWh): 72.419/50.345/189.092/1058.270
High-price hours: 438/8,760 (5.0%)
Jan-Feb 2026: 364/438 high-price hours
Quality: missing=0; UTC duplicates=0; UTC gaps=0; removed=0
```




* Partial months: 2025-06 (13 days); 2026-06 (17 days). Prices use simple hourly means, without load weights.

Prices above the full-sample p95 (> 189.092 USD/MWh) occur in 438 hours; 83.1% fall in Jan-Feb 2026.
The heatmap also shows higher mean prices in late-afternoon/evening hours in several months.
These are descriptive patterns in this rolling year, not causal explanations or a forecasting test.